webserv — Defense Guide

Walkthrough of the evaluation sheet — answers and demos

Before the defense starts

Environment switch (Mac → Linux)

You wrote the server on macOS using kqueue (Server.cpp), but the defense will run on Linux which uses epoll (Server_linux.cpp). The Makefile must compile the right one based on the host OS.

- **Verify on the eval machine:** uname -s should print Linux.
- **In the Makefile,** Server.cpp must be excluded and Server_linux.cpp included on Linux. Confirm this with:

```
grep -E "Server(_linux)?\.cpp" Makefile
```

- *A simple OS-aware Makefile pattern looks like:*

```
UNAME := $(shell uname -s)ifeq ($(UNAME), Darwin)    SRCS += src/Server.cppelse    SRCS +=
src/Server_linux.cppendif
```

Pre-defense checklist (do this together with the evaluator)

- git clone the repo into an empty folder.
- Check the project belongs to you and dzotti.
- Look for hidden aliases: cat ~/.bashrc ~/.zshrc | grep -i webserv
- No malicious scripts hiding behind the test commands.
- Install siege: **sudo apt install siege** (or brew install siege if Homebrew is available on the eval machine).
- Compile cleanly: **make re**. No warnings should appear.
- Run a second **make** — it should output "Nothing to be done" (proves no unnecessary relinking).
- Start the server: **./webserv default.conf**

Mandatory part — Code review and questions

Q1. Explain the basics of an HTTP server

Short answer: HTTP is a stateless request/response protocol on top of TCP. The server listens on a TCP socket, accepts client connections, parses each incoming request (request line + headers + optional body), produces a response (status line + headers + body), and writes it back. After that the connection is either kept alive for more requests (HTTP/1.1 default) or closed.

Walk through your flow: 1) socket() / bind() / listen() on every host:port in the config. 2) epoll watches the listen sockets. 3) On EPOLLIN we accept() a new client and add it to epoll. 4) On client EPOLLIN we recv() into a buffer and feed RequestParser. 5) When the request is complete, ResponseBuilder produces a Response. 6) We swap that into the send buffer and switch the fd to EPOLLOUT. 7) On EPOLLOUT we send() until the buffer is empty. 8) keep-alive → switch back to EPOLLIN; close → removeClient().

Key files to point at: include/Server.hpp (Client struct), src/Server_linux.cpp (event loop), src/Request.cpp (parser), src/Response.cpp (builder).

Q2. Which I/O multiplexing function did you use?

Answer: On Linux we use epoll (epoll_create1, epoll_ctl, epoll_wait). On macOS the same logic uses kqueue. The Makefile picks the right .cpp based on uname.

Show it: open src/Server_linux.cpp and point to:

```
_kq = epoll_create1(0);          // Server constructorepoll_ctl(_kq, ...);             //
addEvent / deleteEvent / modifyEventepoll_wait(_kq, events, MAX_EVENTS, 5000);  // run()
main loop
```

Q3. How does epoll work?

Short answer: epoll is a kernel-side data structure (an "interest list") of file descriptors we want to watch. epoll_ctl adds, modifies, or removes fds with the events we care about (EPOLLIN for "ready to read", EPOLLOUT for "ready to write"). epoll_wait blocks until at least one of those fds is ready, then returns an array of events. We only call recv/send on a fd after epoll told us it is ready, so we never block.

Why epoll over select/poll? epoll is O(1) per ready fd instead of O(n) over all watched fds, scales to thousands of clients without re-passing the full set every call.

Q4. Do you use only one epoll instance? How do accept and read/write coexist?

Answer: Yes, exactly one — _kq is created once in the Server constructor and used for everything: listen sockets, client sockets, and CGI pipes (read and write ends).

How they coexist: Each fd is registered with the events relevant to it. Listen sockets only get EPOLLIN — readiness on a listen fd means "a new connection is waiting", which we accept(). Client sockets are switched between EPOLLIN (parsing a request) and EPOLLOUT (sending a response) using modifyEvent. CGI read pipes are EPOLLIN; CGI write pipes are EPOLLOUT.

Show it: in run(), the dispatch decides what to do based on which map the fd lives in:

```
if (_listen_fds.count(fd))             handleNewConnection(fd);else if
(_cgi_pipe_fds.count(fd))      handleCgiRead(fd);else if (_cgi_write_fds.count(fd))
handleCgiWrite(fd);else if (events[i].events & EPOLLIN)   handleRead(fd);else if
(events[i].events & EPOLLOUT)  handleWrite(fd);
```

Q5. Is epoll in the main loop, watching read AND write at the same time?

Answer: Yes. The single epoll_wait in Server::run() is the only blocking call in the program. Different fds are watched for different events at the same time — the listen socket waits for EPOLLIN, one client may be in EPOLLIN while another is in EPOLLOUT, and a CGI write pipe is in EPOLLOUT in parallel.

> *If they ask you to prove "read AND write at the same time": open two terminals. In one, send a slow upload (POST big file). In the other, send a fast GET. Both will progress in parallel because epoll_wait returns events for both fds in the same iteration.*

Q6. Only one read or one write per client per epoll event

Answer: Correct. handleRead does exactly one recv() of up to 4096 bytes and returns. handleWrite does exactly one send() and returns. We never loop on a single fd inside an event handler.

Show the code path from epoll to read: src/Server_linux.cpp →

```
run() → epoll_wait → loop over events       → handleRead(fd) → ssize_t bytes = recv(fd, buf, 4096, 0);       → returns to event loop
```

Same for write: handleWrite() does one send() with the offset and returns. The next EPOLLOUT event will resume.

Q7. Error handling on recv/send/read/write

Rule: If recv/send/read/write returns -1 OR 0, the client/CGI is removed. We never check errno.

Walk the evaluator through every call site:

```
handleRead     (recv):  if (bytes <= 0) { removeClient(fd); return; }handleWrite    (send): if (bytes <= 0) { removeClient(fd); return; }handleCgiRead  (read):  bytes > 0 → append; else → close pipe + finish CGIhandleCgiWrite (write): bytes > 0 → advance offset; else → close pipe
```

Why <= 0 covers both: 0 means peer closed the connection cleanly (EOF). -1 means an error occurred. Either way, the only safe action is to drop the client and free its fds.

Q8. Both 0 and -1 are checked

The condition `bytes <= 0` catches both. The evaluator may insist you confirm this — just point at any of the four call sites; the comparison is identical.

Q9. Is errno checked anywhere after read/recv/write/send?

Answer: No. The only errno usage in the codebase is inside the event loop after epoll_wait returns -1 (we retry on EINTR, throw otherwise). That is not after a read/write — that is after the multiplexer call itself, which is allowed.

Prove it:

```
grep -n "errno" src/*.cpp include/*.hpp
```

You should see only matches inside Server.cpp / Server_linux.cpp around epoll_wait/kevent — none after recv/send/read/write.

Q10. No read or write on any fd without going through epoll

Sockets and pipes: always epoll-driven. The only recv/send/read/write on socket and pipe fds happen inside handleRead, handleWrite, handleCgiRead, handleCgiWrite, all of which are entered only as a reaction to an epoll event.

Regular files: we use C++ ifstream/ofstream (in serveStaticFile and handleUpload). The subject explicitly allows this — "regular disk files are exempt from the poll/epoll requirement". Mention this preemptively if asked.

Q11. Compilation without re-link issues

```
make fclean && make    # full build, no warningsmake                  # second run →
"make: Nothing to be done for `all`"
```

If the second make rebuilds anything, the dependency tracking is broken. Yours uses .d files for header dependencies and should be clean.

Configuration

All config tests use default.conf. It contains three server blocks: one on 127.0.0.1:8080 (server_name localhost), one virtual host on the same address with server_name example.com, and one on 127.0.0.1:9090 with directory_listing on.

Status codes accuracy

Your getStatusMessage() in Response.cpp covers 200, 201, 204, 301, 302, 400, 403, 404, 405, 413, 500, 501, 504. Every test below pairs a request with the expected code — flag any mismatch immediately, since wrong codes lose points for that whole row.

Multiple servers, different ports

Servers listen on 8080 and 9090. Demo:

```
curl -i http://127.0.0.1:8080/   # should hit the 8080 server (index.html)curl -i
http://127.0.0.1:9090/   # should hit the 9090 server (directory listing on)
```

Multiple servers, different hostnames (virtual hosts)

Your config has two server blocks on 127.0.0.1:8080 — one with server_name localhost, one with example.com. Routing is done in Server::matchServerByHost() based on the Host header.

```
curl -i --resolve example.com:8080:127.0.0.1 http://example.com:8080/curl -i --resolve
localhost:8080:127.0.0.1   http://localhost:8080/
```

Both should return 200 OK from index.html. To make the difference visible, temporarily edit one server's root to point to a different folder before the defense.

Default error page (custom 404)

The config maps `error_page 404 /errors/404.html`. Make sure ./www/errors/404.html exists with custom content.

```
curl -i http://127.0.0.1:8080/this-does-not-exist# Status: 404 Not Found, body: contents of
www/errors/404.html
```

To prove your fallback works too, temporarily move that file aside and re-test — you should still get a 404 with a built-in HTML body.

Client body size limit

The /post_body location has max_body_size 100. Demo both directions:

```
# Within limit (under 100 bytes) → 200 OKcurl -i -X POST -H "Content-Type: text/plain"
--data "short body" \  http://127.0.0.1:8080/post_body# Over limit (longer than 100 bytes)
→ 413 Payload Too Largecurl -i -X POST -H "Content-Type: text/plain" \  --data "$(python3
-c 'print("A"*500)')" \  http://127.0.0.1:8080/post_body
```

Routes pointing to different directories

In default.conf each location has its own root: / → ./www, /directory → ./www/YoupiBanane, /upload → ./www/uploads, /cgi-bin → ./www/cgi-bin. Demo by hitting each:

```
curl -s http://127.0.0.1:8080/index.html | head -3curl -s
http://127.0.0.1:8080/directory/youpi.bad_extension | head -3
```

Default file when requesting a directory

In location /, `index index.html` is set. A request for / returns index.html, not a 403 or directory listing.

```
curl -i http://127.0.0.1:8080/   # serves www/index.html
```

Allowed methods per route

The /upload route accepts GET POST DELETE; / accepts only GET; /post_body accepts only POST. Test the negative cases:

```
# DELETE on / → 405 Method Not Allowedcurl -i -X DELETE http://127.0.0.1:8080/index.html# DELETE on existing /upload/<file> → 204 No Contentcurl -i -X DELETE http://127.0.0.1:8080/upload/test.txt
```

Basic checks

GET request

```
curl -i http://127.0.0.1:8080/         # 200 OK, index.html
```

POST request

```
curl -i -X POST -H "Content-Type: text/plain" \  --data "hello"
http://127.0.0.1:8080/post_body# Returns 200 with the body echoed back (handlePost generic
branch)
```

DELETE request

```
# Create a target first, then delete itecho "delete me" > www/uploads/tmp.txtcurl -i -X
DELETE http://127.0.0.1:8080/upload/tmp.txt# 204 No Content; ls www/uploads/ confirms it is
gone
```

Unknown method (must not crash)

```
curl -i -X PATCH http://127.0.0.1:8080/   # should return 405 (method not allowed for that
location) or 501# Truly junk methods using telnetprintf "BREW / HTTP/1.1\r\nHost:
localhost\r\n\r\n" | nc -q1 127.0.0.1 8080
```

Your parser accepts GET/POST/DELETE/HEAD/PUT and returns 501 Not Implemented for anything else. Either way the server stays up — show top in another terminal to prove the process is still alive after the bad request.

Status codes line up with each test

Quick reference table the evaluator can use:

Test	Expected status
GET / (file exists)	200 OK
GET /missing	404 Not Found
POST /post_body within limit	200 OK
POST /post_body over limit	413 Payload Too Large
DELETE /index.html	405 Method Not Allowed
DELETE /upload/<existing>	204 No Content
DELETE /upload/<missing>	404 Not Found
BREW /	501 Not Implemented
GET /old (redirect)	301 Moved Permanently
Upload (POST multipart) /upload	201 Created

Upload a file and download it back

```
# Create a small test fileecho "defense test $(date)" > /tmp/upload_me.txt# Upload via
multipart/form-datacurl -i -F "file=@/tmp/upload_me.txt" http://127.0.0.1:8080/upload# →
```

```
201 Created, Location: /upload/upload_me.txt# Download it backcurl -i
http://127.0.0.1:8080/upload/upload_me.txt# → 200 OK, body matches the original file
```

handleUpload() in Response.cpp parses the multipart boundary, extracts the filename from Content-Disposition, validates it (no .., no path separators), and writes it to loc.upload_dir.

CGI

CGI runs and returns output

You configure CGI by extension per location: .php → /usr/bin/php-cgi, .py → /usr/bin/python3, .bla → www/cgi-bin/cgi_tester. Demo with the simplest one:

```
# Tiny Python CGI under www/cgi-bin/hello.pycurl -i
"http://127.0.0.1:8080/cgi-bin/hello.py?name=Gio"# Provided cgi_tester (POST)curl -i -X
POST --data-binary @www/index.html \  http://127.0.0.1:8080/directory/youpi.bla
```

CGI runs in the correct directory (chdir)

In Cgi.cpp, the child process calls chdir(getScriptDir()) before execve. So a script can use relative paths and find files next to itself.

Demo: put a script in www/cgi-bin/ that opens a file by relative path:

```
# www/cgi-bin/relpath.py#!/usr/bin/env python3import osprint("Content-Type:
text/plain\r\n\r\n")print("cwd:", os.getcwd())with open("hello.txt") as f:
print(f.read())
```

Place hello.txt next to relpath.py and curl it; the file opens because the working directory is www/cgi-bin/.

CGI with GET and POST

```
# GET — query string is in QUERY_STRING env varcurl -i
"http://127.0.0.1:8080/cgi-bin/hello.py?a=1&b=2"# POST — body is piped on stdin,
CONTENT_LENGTH is setcurl -i -X POST -d "msg=hello" http://127.0.0.1:8080/cgi-bin/hello.py
```

In Cgi.cpp, buildEnv() sets REQUEST_METHOD, QUERY_STRING, CONTENT_LENGTH, CONTENT_TYPE, PATH_INFO, SERVER_PROTOCOL, GATEWAY_INTERFACE, plus all client headers prefixed with HTTP_.

CGI error handling — infinite loop and broken scripts

Infinite loop: CGI_TIMEOUT in Server.hpp is 120s. After that, checkTimeouts() sends SIGKILL, waitpid()s the zombie, and returns 504 Gateway Timeout to the client. The server keeps running.

```
# www/cgi-bin/loop.py#!/usr/bin/env python3while True: passcurl -i --max-time 130
http://127.0.0.1:8080/cgi-bin/loop.py# After ~120s → 504 Gateway Timeout, server is still
alive
```

Broken script: parseCgiOutput() in Cgi.cpp returns 500 with a default body if the CGI produces no output (e.g. crashed before writing anything). If it writes garbage, it is treated as the body with Content-Type text/html.

```
# www/cgi-bin/crash.py#!/usr/bin/env python3import sys; sys.exit(1)curl -i
http://127.0.0.1:8080/cgi-bin/crash.py# → 500 Internal Server Error
```

Server never crashes from CGI misuse

Have a top or htop running in a side terminal during all CGI tests so the evaluator can see the webserv PID stays alive throughout.

Check with a browser

Use Firefox (recommended — DevTools network panel is clear). Open it and navigate to http://127.0.0.1:8080/. Open DevTools (F12) → Network tab.

Inspect request and response headers

- Click any request in the Network tab → Headers panel.
- Request headers from the browser (Host, User-Agent, Accept, etc.) come through your parser.
- Response headers from your server: Date, Server: webserv/1.0, Content-Type, Content-Length, Connection.

Serve a fully static website

Place a small static site under www/ (index.html with a CSS file and an image). Browse to it — every linked asset should load with 200, no 404s in the console.

Wrong URL → 404

Navigate to http://127.0.0.1:8080/does-not-exist. You should see your custom 404 page (www/errors/404.html).

Directory listing

Server on port 9090 has directory_listing on. Visit http://127.0.0.1:9090/ — you should see an HTML index of the www/ directory generated by serveDirectoryListing(). Click the links to descend into subfolders.

Redirect

The /old location redirects to / on the same server.

```
# Browser: navigate to http://127.0.0.1:8080/old# In DevTools you see two requests: GET
/old → 301, then GET / → 200
```

Anything else

Try POSTing through a form, upload a file, refresh repeatedly. The Network tab should show keep-alive connections being reused (look at the Connection ID column).

Port issues

Multiple ports → different sites

Already covered by 8080 vs 9090. To make the difference dramatic for the evaluator, point each server's root at a different folder with a different index.html before the defense.

Same port, same host, same server_name → must fail

Create a quick test config:

```
# bad.confserver { host 127.0.0.1; port 8080; server_name x; location / { root ./www;
methods GET; } }server { host 127.0.0.1; port 8080; server_name x; location / { root ./www;
methods GET; } }./webserv bad.conf# Expected: "Fatal error: Duplicate server combination:
127.0.0.1:8080 (x)"
```

This is exactly what ConfigParser::parse() checks at the bottom — host + port + server_name must be unique across servers. The check is in src/ConfigParser.cpp.

Important nuance: two server blocks on the same host:port with DIFFERENT server_name are virtual hosts and ARE allowed (that's how localhost vs example.com both work on 8080).

Multiple webservs on common ports

If you launch a second ./webserv with the same port already in use, bind() fails:

```
# Terminal 1./webserv default.conf   # works# Terminal 2./webserv default.conf# Expected:
"Fatal error: bind failed on 127.0.0.1:8080"
```

The first instance keeps running and serving requests normally. SO_REUSEADDR is set so a fresh start after a clean shutdown is instant.

Partially broken multi-server config

> *"If one of the configurations isn't functional, why does the server work at all?"*

In your design, the parse step is fail-fast: any malformed server block kills the whole startup. So the situation never arises — you either start cleanly or not at all. State this clearly: "the server only starts if every server block parses correctly; we don't silently ignore broken blocks."

Siege & stress test

Availability ≥ 99.5%

```
# Place a tiny empty page firstecho "" > www/empty.htmlsiege -b -t30s
http://127.0.0.1:8080/empty.html
```

Run for 30 seconds. The summary at the end prints "Availability: ...%". You need at least 99.50%. If it's lower, run again — sometimes the very first connections get refused while siege ramps up.

What to watch:

- Availability ≥ 99.5%
- Failed transactions close to 0
- No "socket: connection reset" floods

No memory leaks (RSS does not grow)

```
# In one terminal: monitor the webserv processtop -p $(pgrep webserv)# Or for a clean
numberwhile true; do ps -o rss= -p $(pgrep webserv); sleep 2; done# In another terminal:
hammer itsiege -b -t60s http://127.0.0.1:8080/
```

Watch the RSS (resident set size). It will grow at the start as buffers are allocated, then plateau. If it keeps climbing linearly with traffic, you have a leak. Yours uses RAII for std::string buffers and explicit close()/erase() for fds and clients — should be flat after warm-up.

Run valgrind for the formal "no leaks" check:

```
valgrind --leak-check=full --show-leak-kinds=all \  --suppressions=/dev/null ./webserv
default.conf# Make a few curl requests, then Ctrl-C# "definitely lost: 0 bytes in 0 blocks"
is the line you want
```

No hanging connections

```
# During and after siege, count established connections to webservss -tn state established
'( sport = :8080 )' | wc -l# Should drop to 0 once siege stops (within TIMEOUT_SEC = 120s)
```

Server::checkTimeouts() walks every client every loop iteration and removes anything idle longer than TIMEOUT_SEC seconds (120s). CGI processes get a separate CGI_TIMEOUT (also 120s) — if a CGI hangs, it gets SIGKILLed and the client gets 504.

Indefinite siege without restart

```
# Long-running testsiege -b -t5m http://127.0.0.1:8080/   # 5 minutes# After it finishes,
the server should still respond:curl -i http://127.0.0.1:8080/
```

No restart needed between siege runs. The fd/memory bookkeeping is symmetric: every accept has a removeClient counterpart.

Bonus part

The bonus is only evaluated if the mandatory part is 100% clean — confirm with the evaluator before claiming bonus points.

Cookies and session management

You did NOT implement a session/cookie system in this version. Be straightforward about it: "We focused on getting the mandatory part fully solid (CGI streaming, virtual hosts, no leaks under siege) and chose not to ship a half-finished session system." Don't bluff — the evaluator will ask you to demo it.

Multiple CGI types

Yes — implemented. default.conf has three CGI types simultaneously, dispatched by file extension:

```
cgi .php /usr/bin/php-cgi;cgi .py  /usr/bin/python3;cgi .bla ./www/cgi-bin/cgi_tester;
```

The dispatch is in ResponseBuilder::matchCgi() — it walks loc.cgi[] and matches by file extension. Demo all three back-to-back to claim the point.

```
curl -i http://127.0.0.1:8080/cgi-bin/info.phpcurl -i
http://127.0.0.1:8080/cgi-bin/hello.pycurl -i http://127.0.0.1:8080/directory/youpi.bla
```

Likely follow-up questions

"Why one recv() per event instead of looping until EAGAIN?"

Two reasons. First, fairness — looping on one client could starve others when there are dozens of fds with data ready. Second, the subject explicitly says "only one read or one write per client per epoll". Looping would violate that even if it would technically work.

"What happens if a client sends a partial request?"

handleRead appends the bytes to client.recv_buffer and returns. RequestParser keeps internal state (PARSE_REQUEST_LINE → PARSE_HEADERS → PARSE_BODY/CHUNKED → PARSE_COMPLETE). Each call to parse() consumes whatever it can and returns. If the request isn't complete, processBufferedRequests does nothing further; we just wait for the next EPOLLIN.

"How do you handle pipelined requests?"

After parsing one complete request, the parser may still have leftover bytes belonging to the next request. We move them back into recv_buffer. After the response is fully sent (handleWrite reaches the end of send_buffer), we call processBufferedRequests again to handle the next one. So one TCP connection can serve many requests in sequence without re-accepting.

"Why two timeout values?"

TIMEOUT_SEC is for idle TCP clients (slow client, dropped connection). CGI_TIMEOUT is for runaway scripts (infinite loops, fork-bombs). They're separate because the appropriate response differs: idle client → just close; CGI hang → SIGKILL the process AND send a 504 to the client.

"Why do you swap into send_buffer instead of assigning?"

client.send_buffer.swap(serialized) is O(1) — it swaps two string headers. Assignment would copy the entire response body, which can be megabytes for large files. Same trick for cgi_body.

"What's the COMPACT_THRESHOLD for?"

When we're sending a 100MB response in 4KB chunks, naively erasing the sent bytes from the front of send_buffer is $O(n^2)$ over the whole transfer. Instead we keep an offset and only compact (erase the prefix) when the offset crosses 64KB. Trades a small amount of memory for a huge speedup on big responses.

"What about virtual hosts on the same port?"

Routing happens in matchServerByHost() during processRequest. We strip the port from the Host header, then walk _configs looking for a server block with a matching server_name on the right port. If none matches, we fall back to the first server bound to that port (the "default server"). That gives you NGINX-style virtual hosting.

"What is early CGI start?"

When a client sends a large POST to a CGI endpoint, we don't wait for the entire body to arrive before forking. As soon as the headers are parsed and we know it's a CGI request with content_length > 0, we fork the CGI immediately and stream body bytes into the CGI's stdin pipe as they arrive over the socket. This avoids buffering huge uploads in memory and matches how real servers like NGINX behave.

One-page cheat sheet

The 5 things the evaluator MUST see in code

- Single epoll instance: epoll_create1 called once in Server constructor.
- epoll watches read AND write on different fds simultaneously.
- One recv()/send() per event handler — no loops.
- All recv/send/read/write check `bytes <= 0` and clean up.
- No errno after read/write — only after epoll_wait (EINTR retry).

Show-stopper grep commands

```
# No errno after read/writegrep -nE "errno" src/*.cpp# Should only show matches near
epoll_wait/kevent# Single epoll_creategrep -nE "epoll_create" src/*.cpp# Should show
exactly one call site# Every recv/send checks <= 0grep -nA2 -E "recv\(|send\("
src/Server_linux.cpp
```

Tests in 60 seconds (run these in front of the evaluator)

```
./webserv default.conf &sleep 1curl -i http://127.0.0.1:8080/
# 200curl -i http://127.0.0.1:8080/missing                          # 404 customcurl -i -X
DELETE http://127.0.0.1:8080/                        # 405curl -i -X POST -d "x"
http://127.0.0.1:8080/post_body          # 200curl -i -X POST --data "$(printf %0500d 0)" \
http://127.0.0.1:8080/post_body                             # 413curl -i
http://127.0.0.1:8080/old                                # 301curl -i http://127.0.0.1:9090/
# 200 + dir listingcurl -i -F "file=@README.md" http://127.0.0.1:8080/upload       #
201siege -b -t10s http://127.0.0.1:8080/                           # ≥99.5%
```